

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА (САМАРСКИЙ УНИВЕРСИТЕТ)»

ИНСТИТУТ ИНФОРМАТИКИ И КИБЕРНЕТИКИ
Кафедра программных систем

А.В. Баландин

Объектно-ориентированное программирование на языке C++

Методические указания к лабораторным работам

САМАРА
2026

Составитель А.В.Баландин

УДК 004.424/432

Баландин А.В. **Объектно-ориентированное программирование на языке C++**. Методические указания к лабораторным работам. - Самар. ун-т. Самара, 2026. 39 с.

Приведены описания и варианты заданий лабораторных работ для практического освоения процедурных и объектно-ориентированных средств программирования языка C++.

В качестве среды программирования рассматривается система Microsoft Visual Studio 10. Однако для выполнения лабораторных работ могут быть использованы и другие версии Microsoft Visual Studio или системы программирования на языке C++.

Методические указания предназначены для студентов, обучающихся по направлению 020302.62 - Фундаментальная информатика и информационные технологии, изучающих языки C/C++.

СОДЕРЖАНИЕ

ПОДГОТОВКА ТЕКСТОВОЙ ЗАГОТОВКИ КОНСОЛЬНОГО ПРИЛОЖЕНИЯ 5

ЛАБОРАТОРНАЯ РАБОТА № 1 8

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ ОБРАБОТКИ СТРОК С ИСПОЛЬЗОВАНИЕМ ФУНКЦИЙ СТАНДАРТНОЙ БИБЛИОТЕКИ ЯЗЫКА С++ В СТИЛЕ ЯЗЫКА С. ... 8

Варианты заданий 9
ВАРИАНТ № 1 9
ВАРИАНТ № 2 10
ВАРИАНТ № 3 10
ВАРИАНТ № 4 10
ВАРИАНТ № 5 10
ВАРИАНТ № 6 10
ВАРИАНТ № 7 11
ВАРИАНТ № 8 11
ВАРИАНТ № 9 11
ВАРИАНТ № 10 11
ВАРИАНТ № 11 12
ВАРИАНТ № 12 12
ВАРИАНТ № 13 12
ВАРИАНТ № 14 13
ВАРИАНТ № 15 13
ВАРИАНТ № 16 13
ВАРИАНТ № 17 13
ВАРИАНТ № 18 14
ВАРИАНТ № 19 14
ВАРИАНТ № 20 14
ВАРИАНТ № 21 15
ВАРИАНТ № 22 15
ВАРИАНТ № 23 15
ВАРИАНТ № 24 15
ВАРИАНТ № 25 15
ВАРИАНТ № 26 16

ЛАБОРАТОРНАЯ РАБОТА № 2 17

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ ПОСЛЕДОВАТЕЛЬНОГО ДОСТУПА К ТЕКСТОВЫМ ФАЙЛАМ С ИСПОЛЬЗОВАНИЕМ ФУНКЦИЙ СТАНДАРТНОЙ БИБЛИОТЕКИ С++ В СТИЛЕ ЯЗЫКА С 17

Варианты заданий 18
ВАРИАНТ № 1 18
ВАРИАНТ № 2 18
ВАРИАНТ № 3 18
ВАРИАНТ № 4 18
ВАРИАНТ № 5 19
ВАРИАНТ № 6 19
ВАРИАНТ № 7 19
ВАРИАНТ № 8 19
ВАРИАНТ № 9 20
ВАРИАНТ № 10 20
ВАРИАНТ № 11 20
ВАРИАНТ № 12 20
ВАРИАНТ № 13 20
ВАРИАНТ № 14 21
ВАРИАНТ № 15 21
ВАРИАНТ № 16 21
ВАРИАНТ № 17 21
ВАРИАНТ № 18 22
ВАРИАНТ № 19 22
ВАРИАНТ № 20 22
ВАРИАНТ № 21 22
ВАРИАНТ № 22 23

ВАРИАНТ № 23.....23
ВАРИАНТ № 24.....23
ВАРИАНТ № 25.....23
ВАРИАНТ № 26.....24

ЛАБОРАТОРНАЯ РАБОТА № 3. 25

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ ПРЯМОГО ДОСТУПА К ЗАПИСЯМ ФАЙЛА С ИСПОЛЬЗОВАНИЕМ ФУНКЦИЙ СТАНДАРТНОЙ БИБЛИОТЕКИ C++ В СТИЛЕ ЯЗЫКА C 25

Варианты заданий 26
ВАРИАНТ № 1.....26
ВАРИАНТ № 2.....26
ВАРИАНТ № 3.....26
ВАРИАНТ № 4.....27
ВАРИАНТ № 5.....27
ВАРИАНТ № 6.....28
ВАРИАНТ № 7.....28
ВАРИАНТ № 8.....28
ВАРИАНТ № 9.....28
ВАРИАНТ № 10.....29
ВАРИАНТ № 11.....29
ВАРИАНТ № 12.....29
ВАРИАНТ № 13.....30
ВАРИАНТ № 14.....30
ВАРИАНТ № 15.....30
ВАРИАНТ № 16.....31
ВАРИАНТ № 17.....31
ВАРИАНТ № 18.....31
ВАРИАНТ № 19.....32
ВАРИАНТ № 20.....32
ВАРИАНТ № 21.....32
ВАРИАНТ № 22.....33
ВАРИАНТ № 23.....33
ВАРИАНТ № 24.....34
ВАРИАНТ № 25.....34

ПРИЛОЖЕНИЕ 1.....35

ЗАДАНИЕ 35

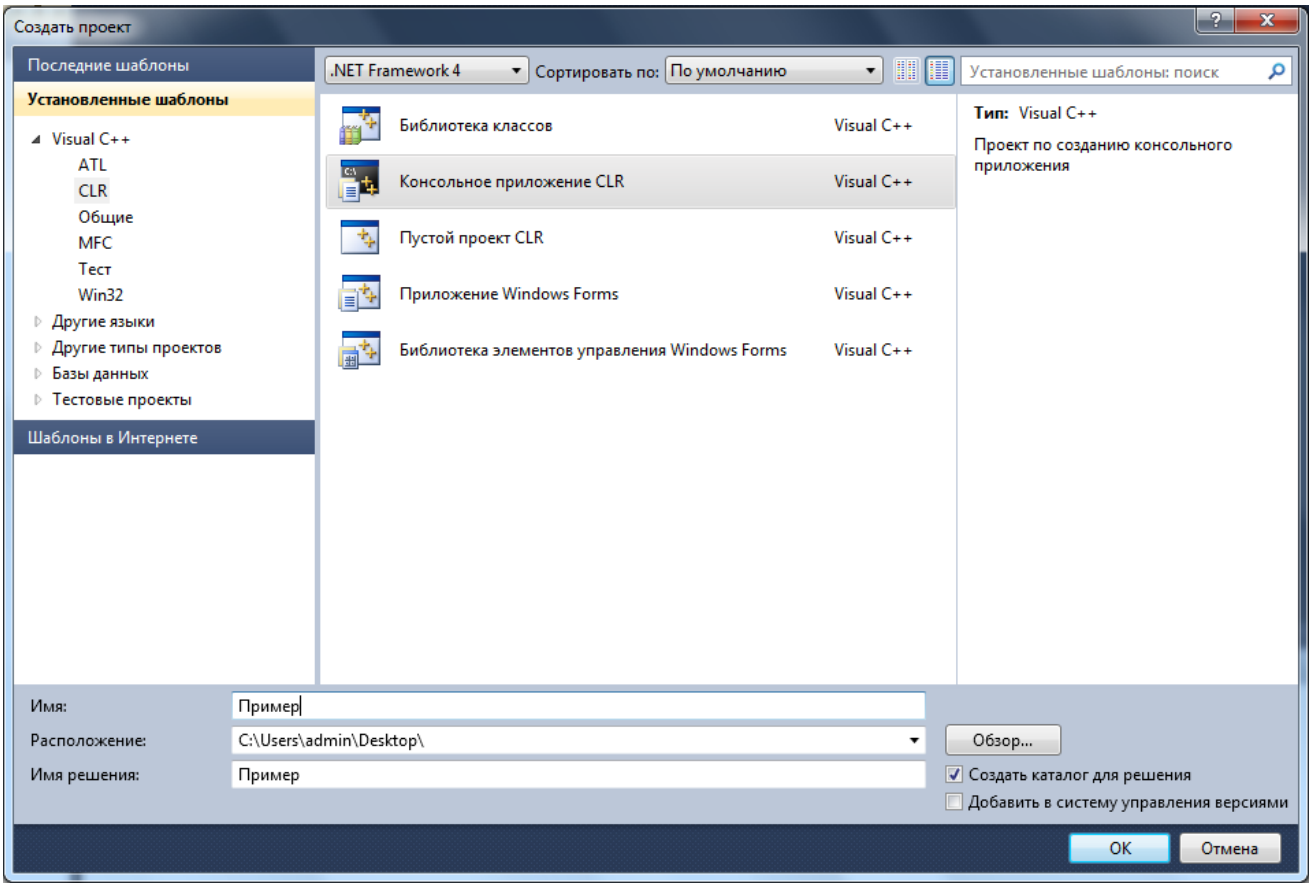
ПРИЛОЖЕНИЕ 2.....37

ЗАДАНИЕ 37

ПРИЛОЖЕНИЕ 3.....39

Подготовка текстовой заготовки консольного приложения

В системе программирования MS Visual Studio различных версий для создания консольного приложения на языке C++ необходимо выполнить команду меню Файл/Создать/Проект…. В открывшемся окне (MS Visual Studio):



необходимо задать имя проекта (например, Пример), место расположения проекта в файловой системе и выбрать тип проекта для создания консольного приложения (выбрать пункт - Консольное приложение CLR). После подтверждения выбора (Ok) автоматически создаётся начальный вид исходного текста приложения:

```
// Пример.cpp: главный файл проекта.


using namespace System;

int main(array<System::String ^> ^args)
{
    Console::WriteLine(L"Здравствуй, мир!");
    return 0;
}
```

Далее следует отредактировать автоматически созданный текст и привести его к виду текстовой заготовки для написания программ в среде языка C++ с использованием стандартных функций в стиле языка C:

```
// Пример.cpp: главный файл проекта.

#define _CRT_SECURE_NO_WARNINGS
#pragma once

#include <iostream>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <Windows.h>

using namespace std;

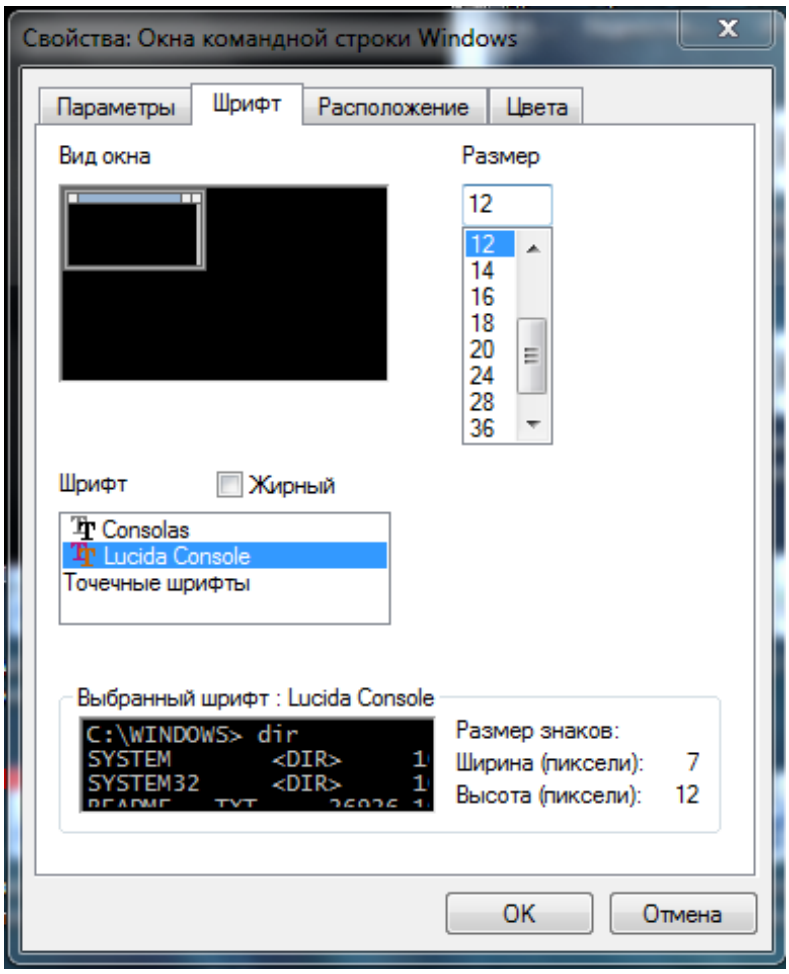
int main(){
system("chcp 1251");//Использовать кириллицу для консольного ввода/вывода

      //Текст
      //программы
      //на языке С

system("pause");//Задержать консоль на экране перед завершением программы
return 0;
}
```

Две команды в теле функции main() имеют технический, «вспомогательный» характер. Их наличие в программном коде приложения связано с особенностями функционирования консольного окна и необходимостью отображать в нем кириллический текст. Речь о командах system("chcp 1251") и system("pause"). Первая переводит консоль в режим использования при вводе/выводе кодировки Windows-1251, а вторая нужна для того, чтобы после выполнения программы консольное окно не закрывалось сразу, а задерживалось на экране до нажатия пользователем любой клавиши (например, Enter или какой-то другой).

При выполнении программы для корректного отображения букв кириллицы при выводе данных в окне консоли необходимо в свойствах окна консоли изменить тип используемого шрифта. Для этого в открытом окне консоли установить курсор мыши на заголовок окна, правой кнопкой мыши вызвать меню окна, выбрать пункт Умолчания, и заменить на панели Шрифт тип шрифта по умолчанию - Точечные шрифты, на любой другой из списка шрифтов (например, Lucida Console):



Полученную в итоге текстовую заготовку следует сохранить для последующего её использования в качестве начальной заготовки при разработках программ лабораторных работ.

Лабораторная работа № 1

Объектно-ориентированное программирование обработки строк с использованием функций стандартной библиотеки языка C++ в стиле языка C.

Цели лабораторной работы:

1. В соответствии с полученным заданием сформулировать и выполнить объектно-ориентированный анализ (ООА) предметной области (ПрО) и построить её информационную модель, состоящую из выделенных и семантически специфицированных свойств и алгоритмов поведения (процедур) информационных объектов (ИО), и связей между информационными объектами.
2. Используя результаты ООА, выполнить объектно-ориентированное проектирование (OOD) выделенных ИО, формализуя их как абстрактные данные с заданным механизмом управления, являющиеся объектами абстрактных типов (классов).

Изучить и использовать стандартные функции языка C++ в стиле языка C для управления:

- строками,
- динамической памятью,
- стандартными внешними устройствами ввода/вывода.

Изученные функции должны использоваться при разработке объектно-ориентированных средств предметной области разрабатываемого в лабораторной работе приложения.

Порядок выполнения

1. Изучить следующие стандартные функции языка C++ в стиле языка C для работы со строками, динамической памятью и устройствами ввода/вывода:

`getchar()`	`strlen()`	`strcmp()`	`strdup()`
`getch()`	`puts()`	`cgets()`	`malloc()`
`gets()`	`strchr()`	`strncmp()`	`free()`
`gets_s()`	`strtok()`	`strstr()`	`realloc()`
`_getch()`	`strrev()`	`strcat()`	
`cgets()`	`strcpy()`	`strspn()`	

2. Осуществить *объектно-ориентированный анализ* (ООАнализ) структуры, представленной в задании предметной области программы - «*абстрактное предложение*», и построить её информационную модель. Обязательной сущностью предметной области программы положить информационный объект типа «Предложение», для которого в соответствии с семантикой задания определить для него свойства и

процедуры управления. Представить и обосновать построенную информационную модель преподавателю. Пример оформления результата объектно-ориентированного анализа и проектирования программы приведён в Приложении 1.

3. Осуществить *объектно-ориентированное проектирование* (ООПроектирование) программы (структуры классов) в соответствии с построенной информационной моделью предметной области.
4. Написать *объектно-ориентированную программу* (ООПрограммирование), реализующую полученный вариант задания в виде консольного приложения в среде программирования на языке C++ с использованием функций стандартной библиотеки языка C++ в стиле языка C.

ЫЧ

5. Продемонстрировать работу программы преподавателю.
6. Предоставить преподавателю письменный отчёт с результатами выполненной лабораторной работы.

Форма отчётности

Для зачёта лабораторной работы преподавателем требуется представить программу и результат её выполнения на компьютере, а также письменный отчёт, включающий в себя:

- Титульный лист (см. Приложение).
- Текст задания.
- Объектно-ориентированный анализ предметной области и построение информационной (объектной) модели.
- Исходный текст программы, включая классы, специфицирующие информационные объекты предметной области.
- Пример выполнения программы (сканы экрана).

Варианты заданий

ВАРИАНТ № 1.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя введённое предложение, построить новое предложение, в котором слова упорядочены по возрастанию в лексикографическом порядке.

ВАРИАНТ № 2.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя введённое предложение, построить новое предложение, в котором слова упорядочены по возрастанию их длин.

ВАРИАНТ № 3.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя введённое предложение, построить новое предложение, в котором слова исходного предложения упорядочены по убыванию значений первых символов слов.

ВАРИАНТ № 4.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя введённое предложение, построить новое предложение, в котором слова упорядочены по возрастанию последних символов.

ВАРИАНТ № 5.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя введённое предложение как исходное, построить новое предложение, в котором символы в словах расположены в обратном порядке по сравнению с исходным предложением.

ВАРИАНТ № 6.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя введённое предложение как исходное, построить новое предложение, в котором, по сравнению с исходным предложением, символы в словах переставлены местами

следующим образом (1-й со 2-м, 3-й с 4-м и т.д.). Если в конце слова парного символа нет, то нечётный символ остаётся на своём месте.

ВАРИАНТ № 7.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя введённое предложение как исходное, построить новое предложение, в котором каждая буква исходного предложения заменена транскрипцией произношения этой буквы, заключённой в квадратные скобки.

ВАРИАНТ № 8.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов, в которых буквы русского алфавита закодированы кодами, подобными кодам азбуки Морзе (придумать ограниченную кодовую таблицу для нескольких букв русского алфавита). Коды букв в словах разделяются символом <*>. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Исходное предложение должно вводиться в программе с помощью функции `getchar()`. Используя исходное закодированного предложение, построить *новое* предложение, представляющее собой расшифрованное исходного предложение, в котором коды заменены соответствующими русскими буквами.

ВАРИАНТ № 9.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Предложение должно быть введено посимвольно с использованием функции `getchar()`. Слова отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть записано прописными буквами в русском алфавите с использованием системы кодирования, в которой букве алфавита поставлен в соответствие 5-значный двоичный код (А - 00000, Б - 00001, В - 00010 и т.д.). Коды в слове отделяются символом <*>. Используя введённое закодированное предложение, построить *новое* предложение, в котором дан перевод исходного предложения.

ВАРИАНТ № 10.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Буквы в словах должны быть записаны в виде кодов с использованием самостоятельно разработанной системы кодирования, в которой прописной букве русского алфавита поставлено

в соответствие некоторое слово (например, А - Арбуз, Б - Булка, В - Вата и т.д.). Коды букв в слове отделяются символом <*>. Используя введённое предложение как исходное и ограниченную кодовую таблицу (составить самостоятельно для семи первых букв алфавита), получить новое предложение, в котором дан перевод исходного закодированного предложения в декодированное предложение, в котором слова составлены из первых букв кодов. Если в исходном предложении встречается отсутствующий в кодовой таблице код буквы, то ему в соответствие ставить знак вопроса - <?>.

ВАРИАНТ № 11.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Предложение должно быть записано прописными буквами английского алфавита. Построить новую строку, в которой дан перевод исходной строки в кодированный текст. В системе кодирования каждой прописной букве английского алфавита ставится в соответствие некоторое английское слово (например, A - apple, B - bull, C - cat и т.д.). Коды букв в закодированном слове отделять символом <*>. Самостоятельно задать кодовую таблицу для семи начальных букв английского алфавита. Если в исходном предложении встречается отсутствующая в кодовой таблице буква, то в качестве соответствующего кода ставить три знака вопроса - <???>.

ВАРИАНТ № 12.

Ввести с клавиатуры две символьные строки длиной не более 40 символов каждая. Строки должны быть введены посимвольно с использованием функции `getchar()`. Строки должны представлять собой произвольные абстрактные предложения, состоящие из слов. Слова отделяются друг от друга как минимум одним символом «пробел». Из исходных строк построить новую строку, в которой нечётные слова берутся из первого введённого предложения, а чётные – из второго. Когда слова в одном из исходных предложений закончатся раньше, чем в другом, его используют повторно с самого начала. Результирующая строка завершается, когда закончатся слова в самом длинном предложении.

ВАРИАНТ № 13.

Ввести с клавиатуры символьную строку длиной не более 80 символов, рассматриваемую как произвольное абстрактное предложение, состоящее из слов, набранных прописными буквами английского алфавита. Слова отделяются друг от друга как минимум одним символом «пробел». Строка должна быть введена посимвольно с использованием функции `getchar()`.

Построить новую строку, в которой содержится перевод исходной строки в кодированный текст. В качестве кодов букв английского алфавита использовать коды, состоящие из точек и тире (подобно азбуке Морзе). Построить кодовую

таблицу для ограниченного подмножества букв английского алфавита (для простоты). Для букв, не вошедших в подмножество, при кодировании использовать специальный код (например, <???>). В закодированных словах коды букв отделять символом <|>.

ВАРИАНТ № 14.

Ввести с клавиатуры символьную строку длиной не более 80 символов. Строка должна быть введена посимвольно с использованием функции `getchar()`. Строка должна представлять собой произвольное абстрактное предложение, состоящее из слов, набранных прописными буквами русского алфавита. Слова отделяются друг от друга как минимум одним символом «пробел».

Построить новую строку, в которой содержится перевод исходной строки в кодированный текст, где каждой букве соответствует 5-значный двоичный код (например, А - 00000, Б - 00001, В - 00010 и т.д.). Построить кодовую таблицу для ограниченного подмножества прописных букв русского алфавита (для простоты). Для букв, не вошедших в подмножество, при кодировании использовать специальный код (например, <???>). В закодированных словах коды букв отделять символом <|>.

ВАРИАНТ № 15.

Ввести с клавиатуры символьную строку длиной не более 80 символов, рассматриваемую как произвольное абстрактное предложение, состоящее из слов. Слова отделяются друг от друга как минимум одним символом «пробел». Строка должна быть введена посимвольно с использованием функции `getchar()`. Используя введённую строку, построить новую строку, в котором слова строки исходного предложения расположены в обратном порядке по сравнению с исходной строкой (первое слово на последнем месте, второе слово на предпоследнем месте, а последнее слово на первом месте).

ВАРИАНТ № 16.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходное предложение, построить новое предложение, в котором слова исходного предложения упорядочены по возрастанию числа *неповторяющихся* символов в соответствующем слове.

ВАРИАНТ № 17.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении

отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходное предложение, построить новое предложение, в котором слова исходного предложения упорядочены по убыванию числа *повторяющихся* символов в соответствующем слове.

ВАРИАНТ № 18.

Ввести с клавиатуры два произвольных абстрактных предложения каждое длиной не более 80 символов, состоящие из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходные предложения, построить новое предложение, в котором на нечётных местах находятся предварительно лексикографически упорядоченные по возрастанию слова из первого предложения, а на чётных местах – лексикографически упорядоченные по убыванию слова из второго предложения. Если слова в одном из исходных предложений закончатся раньше, его используют с начала. Построение результирующего предложения завершается, когда закончатся слова в самом длинном из исходных предложений.

ВАРИАНТ № 19.

Ввести с клавиатуры два произвольных абстрактных предложения каждое длиной не более 80 символов, состоящие из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходные предложения, построить новое предложение, в котором на нечётных местах находятся упорядоченные по возрастанию их длин слова первого предложения, а на чётных местах – упорядоченные по возрастанию их длин слова второго предложения. Если слова в одном из исходных предложений закончатся раньше, его используют с начала. Построение результирующего предложения завершается, когда закончатся слова в самом длинном из исходных предложений.

ВАРИАНТ № 20.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходное предложение, построить новое предложение, составленное из слов исходного предложения, упорядоченных по длине слова, а символы в словах идут в обратном порядке.

ВАРИАНТ № 21.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходное предложение, построить новое предложение, в котором на 1-м месте – последнее слово исходного предложения, на 2-м – первое слово, на 3-м – предпоследнее слово, на 4-м – второе слово и т.д.

ВАРИАНТ № 22.

Ввести с клавиатуры символьную строку длиной не более 80 символов, рассматриваемую как произвольное абстрактное предложение, состоящее из слов. Слова отделяются друг от друга как минимум одним символом «пробел». Строка должна быть введена посимвольно с использованием функции `getchar()`. Используя введённую строку, построить новую строку, в которую включают слова исходного предложения, не содержащие заданный символ (вводится с клавиатуры), лексикографически упорядоченные по убыванию.

ВАРИАНТ № 23.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходное предложение, построить новое предложение, составленное из слов исходного предложения, содержащих заданный символ (вводится с клавиатуры), упорядоченных по убыванию их длин.

ВАРИАНТ № 24.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходное предложение, построить новое предложение, в котором слова расположены в обратном порядке по сравнению с исходным предложением, а символы в словах упорядочены по возрастанию их целых значений.

ВАРИАНТ № 25.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходное

предложение, построить новое предложение, состоящее из слов исходного предложения, которые содержат заданный символ. Символ также вводится с клавиатуры. Символы в словах нового предложения должны быть упорядочены по убыванию их целых значений.

ВАРИАНТ № 26.

Ввести с клавиатуры произвольное абстрактное предложение длиной не более 80 символов, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Предложение должно быть введено с помощью функции `getchar()`. Используя исходное предложение, построить новое предложение, состоящее только из тех слов исходного предложения, которые лексикографически меньше слова введённого с клавиатуры. Символы в словах результирующего предложения должны иметь обратный порядок следования.

Лабораторная работа № 2

Объектно-ориентированное программирование последовательного доступа к текстовым файлам с использованием функций стандартной библиотеки C++ в стиле языка C

Цель лабораторной работы: изучить и практически освоить при разработке программы использование указателей на функции, а также стандартные функции языка C++ в стиле языка C для работы с файлами в текстовом режиме, стандартные функции поиска и сортировки элементов в массивах произвольных типов, а также правила преобразования в языке C++ стандартных типов при использовании в выражениях разнотипных данных.

Порядок выполнения

1. Изучить следующие стандартные функции языка C++ в стиле языка C:

`fopen()`	`fputc()`	`fgetc()`	`fputs()`	`fgets()`
`printf()`	`scanf()`	`fprintf()`	`fscanf()`	`sprintf()`
`qsort()`	`lfind()`	`lsearch()`	`bsearch()`	`sscanf()`
`feof()`	`ferror()`			

2. Осуществить *объектно-ориентированный анализ* (**OOA**) структуры, представленной в задании предметной области программы - «*Текстовый_файл*», и построить её информационную структуру, моделирующую предметную область посредством информационных объектов-"сущностей" и объектов-"связей". Представить и обосновать построенную информационную модель преподавателю. Пример оформления результата объектно-ориентированного анализа и проектирования программы приведён в Приложении 2.
3. Осуществить *объектно-ориентированное проектирование* (**OOD**). Определить абстрактные типы для информационных объектов: типы свойств, типы процедур.
4. Написать объектно-ориентированную программу, реализующую полученный вариант задания в виде консольного приложения в среде Visual Studio или любой другой среде программирования на языке C++.

При написании программы необходимо где целесообразно, применить изученные ранее функции стандартной библиотеки языка C++ в стиле C, а также стандартные функции в стиле языка C для работы с текстовыми файлами.

1. Продемонстрировать работу программы преподавателю.
2. Предоставить преподавателю отчёт с результатами выполненной работы.

Форма отчётности

Для отчёта по лабораторной работе требуется представить результат выполнения программы на компьютере и отчёт в установленной форме, включающий в себя:

- Титульный лист (см. Приложение).

- Текст задания.
- Объектная модель предметной области.
- Исходный текст программы.
- Изображение окна работы программы
- Изображения окна текстового редактора с содержимым файлов

Варианты заданий

ВАРИАНТ № 1.

Написать программу, которая создаёт в текущем каталоге текстовый файл с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. После создания исходного файла необходимо создать новый текстовый файл с именем, вводимым с клавиатуры, в который строки исходного файла поместить *по возрастанию их длин*. Для этого содержимое исходного файла следует полностью переписать в оперативную память, после чего использовать для их сортировки стандартную функцию `qsort()`.

ВАРИАНТ № 2.

Написать программу, которая создаёт в текущем каталоге текстовый файл с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. После создания исходного файла необходимо по введённому с клавиатуры порядковому номеру строки в файле вывести её содержимое на экран. В случае отсутствия в файле строки с указанным номером, вывести сообщение об ошибке.

ВАРИАНТ № 3.

Написать программу, которая создаёт в текстовом режиме файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. После ввода последней строки файл закрывается. Построить в оперативной памяти массив записей (структур) вида <*номер_строки*, *количество_различных_символов*>, анализируя строки файла. Отсортировать полученный массив по возрастанию значений поля "*количество_различных_символов*", результирующий массив вывести на экран. Для сортировки элементов массива использовать стандартную функцию языка C `qsort()`.

ВАРИАНТ № 4.

Написать программу, которая создаёт в текстовом режиме файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. После ввода последней строки файл закрывается. По введённой с клавиатуры длине строки извлечь из файла все

строки, длиннее заданной величины, и поместить их в другой текстовый файл в текущем каталоге с заданным именем. В файле с прежним именем оставить только строки короче или равные заданной длине.

ВАРИАНТ № 5.

Написать программу, которая открывает в текстовом режиме произвольный текстовый файл в указанном каталоге, в соответствии с его полным именем, вводимым с клавиатуры. Далее программа должна по указанному символу подсчитывать сколько раз он встречается в тексте, содержащимся в файле, и сколько в тексте строк, которые *не содержат* этот символ.

ВАРИАНТ № 6.

Написать программу, которая создаёт в текстовом режиме файл с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. После ввода последней строки файл закрывается. Разделить данный файл на два таких текстовых файла, что в один попадут все строки, начинающиеся на заданный символ, а в другой - все остальные строки. Имена новых файлов и символ вводятся с клавиатуры.

ВАРИАНТ № 7.

Написать программу, которая создаёт в текстовом режиме файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его произвольными предложениями, которые также вводятся с клавиатуры. Слова в предложениях отделяются как минимум одним пробелом. После ввода последнего предложения файл закрывается. Построить в оперативной памяти массив записей (структур) вида <*номер_предложения*, *количество_слов*>, анализируя предложения файла. Отсортировать полученный массив по возрастанию значений поля "*количество_слов*". Результирующий массив вывести на экран. Для сортировки элементов массива использовать стандартную функцию в стиле языка C - `qsort()`.

ВАРИАНТ № 8.

Написать программу поиска в существующем текстовом файле, полное имя которого вводится с клавиатуры, таких строк, которые содержат заданную подстроку, вводимую с клавиатуры. Найденные в процессе поиска строки размещаются в динамически формируемом массиве строк переменной длины, расположенных в динамически выделенной под строку памяти, строго равной длине строки. В итоге размер массива должен быть точно равен количеству найденных строк (см. функцию `realloc()`). Отсортировать лексикографически полученный массив, используя стандартную функцию в стиле языка C - `qsort()`, и вывести на экран. Результирующий массив строк сохранить в файле с заданным с клавиатуры именем.

ВАРИАНТ № 9.

Написать программу, которая для заданного текстового файла, имя которого вводится с клавиатуры, строит в оперативной памяти массив записей (структур) вида <*длина_строки*, *количество_строк*>, анализируя прочитанные строки файла. Отсортировать полученный массив по возрастанию длин строк, результирующий массив вывести на экран. Для сортировки элементов массива использовать стандартную функцию в стиле языка С - `qsort()`.

ВАРИАНТ № 10.

Написать программу, которая открывает в текстовом режиме произвольный текстовый файл в указанном каталоге, в соответствии с его полным именем, вводимым с клавиатуры. Далее программа должна распечатать все строки файла, имеющие длину, которая задается с клавиатуры. Строки выводятся в порядке их следования в файле.

ВАРИАНТ № 11.

Написать программу, которая создаёт в текстовом режиме файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его словами, вводимыми с клавиатуры, завершающимися символом '\n'. После ввода последнего слова файл закрывается. Поместить в новый текстовый файл слова исходного файла, начинающиеся с символов, лексикографически меньших, чем заданный символ. В исходном файле оставить слова, начинающиеся с символов, лексикографически больших, чем заданный символ. Имя второго файла и символ также вводятся с клавиатуры. В двух результирующих файлах слова должны быть упорядочены по алфавиту. При написании программы для сортировки используется стандартная функция `qsort()`.

ВАРИАНТ № 12.

Написать программу, которая создает в текстовом режиме файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. После ввода последней строки файл закрывается. Далее программа должна распечатать все строки файла, заканчивающиеся на заданный символ, вводимый с клавиатуры. Строки выводятся в порядке их следования в исходном файле.

ВАРИАНТ № 13.

Написать программу, которая для заданного текстового файла, имя которого вводится с клавиатуры, строит в оперативной памяти массив записей (структур) вида <*длина_строки*, *количество_строк*>, анализируя строки файла. Отсортировать полученный массив по возрастанию значений поля "*количество_строк*", результирующий массив вывести на экран. Для поиска и

сортировки элементов массива использовать стандартные функции в стиле языка C - `lsearch()` и `qsort()` соответственно.

ВАРИАНТ № 14.

Написать программу, которая создаёт в текстовом режиме файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. После ввода последней строки файл закрывается. Необходимо создать новый текстовый файл с именем, вводимым с клавиатуры, в который поместить строки исходного файла, имеющие заданную длину, вводимую с клавиатуры. Строки в результирующий файл помещаются в лексикографическом порядке Для сортировки строк должна быть использована стандартная функция в стиле языка C - `qsort()`, для чего предварительно содержимое исходного файла полностью переписывается в оперативную память.

ВАРИАНТ № 15.

Написать программу, которая создаёт текстовый файл и заполняет его словами, вводимыми с клавиатуры, завершающимися символом '\n'. После ввода последнего слова файл закрывается. Создать новый файл с именем, вводимым с клавиатуры, в котором слова исходного файла располагаются в лексикографическом порядке. Для сортировки слов содержимое исходного файла полностью переписывается в оперативную память. При написании программы должна быть использована стандартная функция в стиле языка C - `qsort()`.

ВАРИАНТ № 16.

Написать программу, которая создаёт текстовый файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. После ввода последней строки файл закрывается. Создать новый текстовый файл с именем, вводимым с клавиатуры, в который поместить все строки, взятые из исходного файла, содержащие заданный символ. Символ вводится с клавиатуры. В результирующем файле строки должны следовать по возрастанию их длин. Сортировку строк перед записью в файл выполнить в оперативной памяти с помощью стандартной функции в стиле языка C - `qsort()`.

ВАРИАНТ № 17.

Написать программу, которая создаёт файл и заполняет его с помощью функции `fprintf()` записями вида <"*строка*", *длина_строки*>, вводимыми с клавиатуры. Поле "*строка*" заполняется произвольной строкой. После ввода последней записи и закрытия файла программа должна создать ещё один файл, в котором записи исходного файла помещаются в порядке возрастания значений поля "*длина_строки*". Для сортировки содержимое исходного файла полностью

переписывается в оперативную память и интерпретируется как массив записей. Для сортировки записей в массиве использовать стандартную функцию в стиле языка C - `qsort()`.

ВАРИАНТ № 18.

Написать программу, которая открывает в текстовом режиме произвольный текстовый файл. Полное имя файла вводится с клавиатуры. С клавиатуры также задаётся длина строки. Программа должна распечатать все строки файла, имеющие длину, не равную заданной. Строки выводятся в лексикографическом порядке. Найденные строки переписываются в оперативную память и интерпретируются как массив записей. Для сортировки записей в массиве должна быть использована стандартная функция в стиле языка C - `qsort()`.

ВАРИАНТ № 19.

Написать программу, которая создаёт в текстовом режиме файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его произвольными предложениями, которые также вводятся с клавиатуры. После ввода последнего предложения файл закрывается. Построить в оперативной памяти массив записей (структур) вида <*количество_слов_в_предложении*, *количество_предложений*>, анализируя строки файла. Отсортировать полученный массив по убыванию значений поля "*количество_предложений*", результирующий массив вывести на экран. Для сортировки элементов массива использовать стандартную функцию в стиле языка C `qsort()`.

ВАРИАНТ № 20.

Написать программу, которая для заданного текстового файла, имя которого вводится с клавиатуры, строит в оперативной памяти массив записей (структур) вида <*номер_строки*, *количество_символов_имеющих_повторы*>, анализируя строки файла. Отсортировать полученный массив по возрастанию значений поля "*количество_символов_имеющих_повторы*", результирующий массив вывести на экран. Для сортировки элементов массива использовать стандартную функцию в стиле языка C `qsort()`.

ВАРИАНТ № 21.

Написать программу, которая создаёт в текстовом режиме файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. С клавиатуры также задается символ. Программа строит в оперативной памяти массив записей (структур) вида <"*строка*", *количество_вхождений_символа*>, сортирует массив записей лексикографически по возрастанию поля "*строка*" с помощью стандартной функции в стиле языка C `qsort()` и сохраняет результирующий массив в новый файл с именем, задаваемым с клавиатуры.

ВАРИАНТ № 22.

Написать программу, которая создаёт в текстовом режиме файл в текущем каталоге с именем, вводимым с клавиатуры, и заполняет его произвольными предложениями, которые также вводятся с клавиатуры. После ввода последнего предложения файл закрывается. Далее программа должна распечатать все предложения файла с указанием количества слов предложения, содержащих заданный с клавиатуры символ. Если ни в одном слове предложения символ не встречается, то предложение на экран не выводится. Предложения выводятся в порядке их следования в файле.

ВАРИАНТ № 23.

Написать программу, которая открывает в текстовом режиме исходный текстовый файл, содержащий произвольные предложения, заканчивающиеся управляющим символом '\n'. Полное имя исходного файла вводится с клавиатуры.

Создать новый файл, имя которого также вводится с клавиатуры, и записать в него все слова (как строки, заканчивающиеся управляющим символом '\n') всех предложений исходного файла, содержащие в себе заданную подстроку, введённую с клавиатуры. Слова-строки в созданном новом файле должны располагаться в лексикографическом порядке. Для сортировки слов их предварительно переписать в динамическую память и отсортировать, используя стандартную функцию в стиле языка C - `qsort()`.

ВАРИАНТ № 24.

Написать программу поиска в существующем текстовом файле, полное имя которого вводится с клавиатуры, таких строк, которые содержат заданную подстроку, вводимую с клавиатуры. Найденные в процессе поиска строки переписываются в динамическую память. Отсортировать строки в памяти по убыванию их длин, используя стандартную функцию в стиле языка C - `qsort()`. Отсортированные строки сохранить в созданном новом файле с именем, заданным с клавиатуры, и вывести на экран.

ВАРИАНТ № 25.

Написать программу, которая создаёт в текущем каталоге текстовый файл с именем, вводимым с клавиатуры, и заполняет его произвольными предложениями, которые также вводятся с клавиатуры.

После создания исходного файла необходимо по введённому с клавиатуры порядковому номеру предложения в файле вывести количество слов в предложении на экран. В случае отсутствия в файле предложения с указанным номером, вывести сообщение об ошибке.

ВАРИАНТ № 26.

Написать программу, которая создаёт в текущем каталоге текстовый файл с именем, вводимым с клавиатуры, и заполняет его произвольными строками, которые также вводятся с клавиатуры. После создания исходного файла необходимо создать новый текстовый файл с именем, вводимым с клавиатуры, в который строки исходного файла поместить *в обратном порядке*. Для этого содержимое исходного файла следует предварительно полностью переписать в оперативную память.

Лабораторная работа № 3.

Объектно-ориентированное программирование прямого доступа к записям файла с использованием функций стандартной библиотеки C++ в стиле языка C

Цель лабораторной работы: изучить и практически использовать стандартные функции языка C для работы с файлами, открытыми в двоичном режиме.

Порядок выполнения

1. Изучить следующие стандартные функции языка C:
   `fopen()` `fclose()` `fwrite()`
   `fread()` `fseek()` `ftell()`
2. На основании текста задания провести объектно-ориентированный анализ (ООА) предметной области (ПрО) программы и построить её информационную модель, состоящую из информационных объектов-сущностей и объектов-связей между ними. Согласовать информационную модель с преподавателем.
3. На основании полученной информационной модели ПрО выполнить объектно-ориентированное проектирование (ООП) программы, разработав соответствующие информационным объектам ПрО классы и укрупнённый алгоритм управления информационными объектами в соответствии с заданием.
4. Написать в среде программирования на языке C++ объектно-ориентированную программу, использующую функции стандартной библиотеки в стиле языка C, реализующую полученный вариант задания в виде консольного приложения.

При написании программы необходимо где целесообразно, применить изученные ранее функции стандартной библиотеки языка C++ в стиле C, а также стандартные функции в стиле языка C для работы с двоичными файлами.

Форма отчётности

Для отчёта по лабораторной работе требуется представить результат выполнения программы на компьютере и отчёт в установленной форме, включающий в себя:

- Титульный лист (см. Приложение).
- Текст задания.
- Объектная модель предметной области.
- Исходный текст программы.

Варианты заданий

ВАРИАНТ № 1

Разработать информационную базу, состоящую из трех файлов: SENSOR.DAT, SUBSYS.DAT, LINK.IDX. Файл SENSOR.DAT содержит записи типа struct {unsigned shifr; //шифр датчика; char name[41]; //название датчика}. Файл SUBSYS.DAT содержит записи типа struct {unsigned shifr; //шифр подсистемы; char name[41]; //название подсистемы}. Файл LINK.IDX содержит записи типа struct {unsigned long sensor_idx; //индекс записи в файле SENSOR.DAT; unsigned long subsys_idx; //индекс записи в файле SUBSYS.DAT}. В качестве индексов используются значения позиций, с которых начинаются записи в соответствующих файлах. Записи в SENSOR.DAT и в LINK.IDX взаимно-однозначно соответствуют друг другу. Файл LINK.IDX показывает, какие датчики принадлежат каким подсистемам. Программа должна обеспечивать выполнение следующих функций: 1) добавление записи в SENSOR.DAT; 2) добавление записи в SUBSYS.DAT; 3) вывод на экран по шифру датчика названия подсистемы, которой датчик принадлежит.

ВАРИАНТ № 2

Разработать информационную базу, состоящую из трёх файлов: SENSOR.DAT, SUBSYS.DAT, LINK.IDX. Файл SENSOR.DAT содержит записи типа struct {unsigned shifr; //шифр датчика; char name[41]; //название датчика}. Файл SUBSYS.DAT содержит записи типа struct {unsigned shifr; //шифр подсистемы; char name[41]; //название подсистемы}. Файл LINK.IDX содержит записи типа struct {unsigned long sensor_idx; //индекс датчика в файле SENSOR.DAT; unsigned long subsys_idx; //индекс подсистемы в файле SUBSYS.DAT}. В качестве индексов используются значения позиций, с которых начинаются записи в соответствующих файлах. Записи в SENSOR.DAT и в LINK.IDX взаимно-однозначно соответствуют друг другу. Файл LINK.IDX показывает, какие датчики принадлежат каким подсистемам. Программа должна обеспечивать выполнение следующих функций:
1) добавление нового датчика с указанием подсистемы, которой принадлежит датчик; 2) добавление новой подсистемы; 3) вывод на экран по шифру подсистемы всех датчиков, которые в ней находятся.

ВАРИАНТ № 3

Реализовать информационную базу, состоящую из трех файлов: FIO.DAT - содержит в виде символьных строк, оканчивающихся <\n>, список фамилий; DOLGH.DAT - содержит в виде символьных строк наименования должностей, которые не повторяются; LINK.IDX - содержит записи вида struct {unsigned long name_idx; //индекс записи в файле FIO.DAT; unsigned long post_idx;

//индекс записи в файле DOLGH.DAT}, связывающие между собой фамилию и должность. В качестве индексов используются значения позиций, с которых начинаются записи в соответствующих файлах. Записи в FIO.DAT и в LINK.IDX взаимно-однозначно соответствуют друг другу. Программа должна обеспечивать выполнение следующих функций: 1) включение новой фамилии с указанием должности, 2) добавление новой должности, 3) по указанной фамилии вывод на экран занимаемой должности,

ВАРИАНТ № 4

Реализовать информационную базу, состоящую из трех файлов:

FIO.DAT - содержит в виде символьных строк, оканчивающихся <\n>, список фамилий;

DOLGH.DAT - содержит в виде символьных строк наименования должностей, которые не повторяются;

LINK.IDX - содержит записи вида:

```
struct {unsigned long name_idx; //индекс записи в файле FIO.DAT;
        unsigned long post_idx; //индекс записи в файле DOLGH.DAT
},
```

связывающие фамилии в FIO.DAT и должности в DOLGH.DAT. В качестве индекса используется значение позиции записи от начала файла, с которой запись располагается в соответствующем файле. Записи в FIO.DAT и в LINK.IDX взаимнооднозначно соответствуют друг другу. Программа должна обеспечивать выполнение следующих функций:

1) включение в информационную базу новой фамилии с указанием должности;
2) добавление новой должности;
3) по указанному порядковому номеру в индексном файле вывод на экран фамилии и должности в виде:
   Сотрудник <фамилия> - **должность** <должность>.

ВАРИАНТ № 5

Реализовать информационную базу, состоящую из трех файлов: FIO.DAT - содержит в виде символьных строк, оканчивающихся <\n>, список фамилий; DOLGH.DAT - содержит в виде символьных строк наименования должностей, которые не повторяются; LINK.IDX - содержит записи вида `struct {unsigned long name_idx; //индекс записи в файле FIO.DAT; unsigned long post_idx; //индекс записи в файле DOLGH.DAT}`, связывающие между собой фамилию и должность. В качестве индексов используются значения позиций, с которых начинаются записи в соответствующих файлах. Записи в FIO.DAT и в LINK.IDX взаимно-однозначно соответствуют друг другу. Программа должна обеспечивать выполнение следующих функций: 1) включение новой фамилии с указанием должности, 2) добавление новой должности, 3) по указанной должности вывод на экран всех соответствующих фамилий.

ВАРИАНТ № 6

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему произвольные символьные строки, заканчивающиеся <\n>. Программа должна обеспечивать выполнение следующих действий: 1) формирование файла строк и создание по его завершению индексного файла с записями вида <unsigned long idx>, в котором индексы располагаются в порядке возрастания длин соответствующих строк исходного файла (использовать функцию qsort); 2) нахождение и вывод на экран строки по заданному значению её длины (использовать функцию двоичного поиска bsearch).

ВАРИАНТ № 7

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему произвольные символьные строки, заканчивающиеся <\n>. Программа должна обеспечивать выполнение следующих действий: 1) формирование файла записей и создание по его завершению индексного файла с записями вида <unsigned long idx>, в котором индексы располагаются в порядке следования строк в исходном файле; 2) на основе исходного и индексного файлов сформировать файл с указанным именем, содержащий те же записи, но в обратном порядке.

ВАРИАНТ № 8

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему произвольные символьные строки, заканчивающиеся <\n>. Программа должна обеспечивать выполнение следующих действий: 1) формирование файла записей и создание по его завершению индексного файла с записями вида <unsigned long idx>, в котором индексы располагаются в порядке лексикографического возрастания строк исходного файла (использовать функцию в стиле языка С qsort); 2) нахождение и вывод на экран по заданному номеру записи в индексном файле всех лексикографически старших строк исходного файла.

ВАРИАНТ № 9

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему записи структуры вида: struct {unsigned kard; //номер зачетной книжки; char name[21]; //фамилия студента}. Программа должна обеспечивать выполнение следующих функций: 1) формирование файла записей и создание по его завершению двух индексных файлов, в первом индексы располагаются в порядке возрастания поля kard соответствующих записей, а во втором – в порядке лексикографического возрастания поля name соответствующих записей (для упорядочения индексов

использовать функцию в стиле языка C `qsort`); 2) нахождение и вывод на экран фамилии студента по заданному номеру зачетной книжки (использовать функцию `bsearch` двоичного поиска в стиле языка C); 3) вывод на экран упорядоченного по алфавиту списка всех фамилий студентов.

ВАРИАНТ № 10

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему записи структуры вида: `struct {unsigned kard; //номер` зачетной книжки; `char name[21]; //фамилия студента}`. Программа должна обеспечивать выполнение следующих функций: 1) формирование файла записей и создание по его завершению двух индексных файлов, в первом индексы располагаются в порядке возрастания поля `kard` соответствующих записей, а во втором – в порядке возрастания длин поля `name` соответствующих записей (для упорядочения индексов использовать функцию `qsort` в стиле языка C); 2) нахождение и вывод на экран фамилии студента по заданному номеру зачетной книжки (использовать функцию двоичного поиска `bsearch` в стиле языка C); 3) вывод на экран упорядоченного по длинам списка всех фамилий студентов.

ВАРИАНТ № 11

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему записи структуры вида: `struct {unsigned kard; //номер` зачётной книжки; `char name[21]; //фамилия студента}`. Программа должна обеспечивать выполнение следующих функций:
1) формирование файла записей и создание по его завершению двух индексных файлов, в первом индексы располагаются в порядке возрастания поля `kard` соответствующих записей, а во втором – в порядке лексикографического возрастания поля `name` соответствующих записей (для упорядочения индексов использовать функцию `qsort` в стиле языка C); 2) нахождение и вывод на экран фамилии студента по заданному номеру зачетной книжки (использовать функцию двоичного поиска `bsearch` в стиле языка C); 3) формирование файла, содержащего записи о студентах, являющихся однофамильцами студента, найденного по заданному номеру зачётной книжки.

ВАРИАНТ № 12

Реализовать информационную базу, состоящую из трёх файлов: FIO.DAT - содержит в виде символьных строк, оканчивающихся <\n>, список фамилий спортсменов; `SPORT.DAT` - содержит в виде символьных строк наименования видов спорта, которые не повторяются; `LINK.IDX` - содержит записи вида `struct {unsigned long name_idx; //индекс записи в файле FIO.DAT; unsigned long sport_idx; //индекс записи в файле SPORT.DAT}`, связывающие между собой фамилию и вид спорта. В качестве индексов

используются значения позиций, с которых начинаются записи в соответствующих файлах. Записи в FIO.DAT и в LINK.IDX взаимно-однозначно соответствуют друг другу. Программа должна обеспечивать выполнение следующих функций: 1) включение новой фамилии спортсмена с указанием вида спорта, 2) добавление нового вида спорта, 3) по заданной фамилии спортсмена вывод на экран вида спорта, которым тот занимается, если заданная фамилия соответствует нескольким видам спорта, то они все выводятся на экран.

ВАРИАНТ № 13

Реализовать информационную базу, состоящую из трёх файлов: FIO.DAT - содержит в виде символьных строк, оканчивающихся <\n>, список фамилий спортсменов; SPORT.DAT - содержит в виде символьных строк наименования видов спорта, которые не повторяются; LINK.IDX - содержит записи вида struct {unsigned long name_idx; //индекс записи в файле FIO.DAT; unsigned long sport_idx; //индекс записи в файле SPORT.DAT}, связывающие между собой фамилию и вид спорта. В качестве индексов используются значения позиций, с которых начинаются записи в соответствующих файлах. Записи в FIO.DAT и в LINK.IDX взаимно-однозначно соответствуют друг другу. Программа должна обеспечивать выполнение следующих функций: 1) включение новой фамилии спортсмена с указанием вида спорта, 2) добавление нового вида спорта, 3) по заданному виду спорта вывод на экран фамилий всех спортсменов, занимающихся данным видом спорта.

ВАРИАНТ № 14

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему записи структуры вида: struct {unsigned shifr; //шифр датчика; char name[21]; //название датчика}. Программа должна обеспечивать выполнение следующих функций: 1) формирование файла записей и создание по его завершению двух индексных файлов, в первом индексы располагаются в порядке возрастания поля shifr соответствующих записей, а во втором – в порядке возрастания длин поля name (для упорядочения индексов использовать функцию qsort); 2) нахождение и вывод на экран названия датчика по заданному значению шифра датчика (использовать функцию двоичного поиска bsearch); 3) вывод на экран упорядоченного по длинам списка названий датчиков.

ВАРИАНТ № 15

Написать программу, которая поддерживает индексированный доступ к файлу базы данных с информацией о датчиках типа:

```
struct{unsigned shifr; //шифр датчика;
       char name[21]; //название датчика}.
```

Программа должна обеспечивать выполнение следующих действий:

1) формирование файла базы данных с заданным именем и двух индексных файлов для прямого доступа к записям, в первом индексы должны быть отсортированы в порядке возрастания поля `shifr` соответствующих записей, а во втором – в порядке лексикографического возрастания поля `name` соответствующих записей (для сортировки содержимое индексных файлов переписывается в буфер динамически выделяемой оперативной памяти, и используется функция `qsort`);

2) нахождение и вывод на экран названия датчика по заданному пользователем значению шифра датчика (использовать функцию двоичного поиска `bsearch` в стиле языка С);

3) вывод на экран лексикографически упорядоченного по названиям списка датчиков.

ВАРИАНТ № 16

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему записи структуры вида `struct {unsigned length; //длина строки; char *str; //символьная строка переменной длины}`. Программа должна обеспечивать выполнение следующих функций: 1) формирование файла записей и создание по завершению индексного файла с записями вида `struct {unsigned long length; //длина строки в файле записей; unsigned long idx; //индекс записи в файле}`, в котором индексные записи располагаются в порядке, возрастания поля `length` (для упорядочения индексов использовать функцию `qsort`); 2) на основе сформированных исходного и индексного файлов формирование файла с указанным именем, содержащего записи вида <`char *str`><`\n`>, упорядоченные по их длинам.

ВАРИАНТ № 17

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему произвольные символьные строки, заканчивающиеся <`\n`>. Программа должна обеспечивать выполнение следующих действий: 1) формирование файла строк и создание по его завершению индексного файла с записями вида <`unsigned long idx`>, в котором индексы располагаются в порядке возрастания длин соответствующих строк исходного файла (использовать функцию `qsort` в стиле языка С); 2) на основе сформированных исходного и индексного файлов формирование файла с указанным именем, содержащего строки исходного файла, упорядоченные по убыванию длины.

ВАРИАНТ № 18

Написать программу, которая поддерживает индексированный доступ к файлу, содержащему произвольные символьные строки, заканчивающиеся <`\n`>.

Программа должна обеспечивать выполнение следующих действий: 1) формирование файла записей и создание по его завершению индексного файла с записями вида <unsigned long idx>, в котором индексы располагаются в порядке лексикографического возрастания строк исходного файла (использовать функцию qsort в стиле языка С); 2) нахождение и вывод на экран по заданному номеру записи в индексном файле всех лексикографически меньших строк исходного файла в порядке, обратном порядку их следования в исходном файле.

ВАРИАНТ № 19

Реализовать информационную базу, состоящую из трёх файлов: FIO.DAT - содержит в виде символьных строк, оканчивающихся <\n>, список фамилий студентов; GROUP.DAT - содержит в виде символьных строк номера студенческих групп, которые на повторяются; LINK.IDX - содержит записи вида struct {unsigned long name_idx; //индекс записи в файле FIO.DAT; unsigned long group_idx; //индекс записи в файле GROUP.DAT}, связывающие между собой фамилию и группу. В качестве индексов используются значения позиций, с которых начинаются записи в соответствующих файлах. Записи в FIO.DAT и в LINK.IDX взаимно-однозначно соответствуют друг другу. Программа должна обеспечивать выполнение следующих функций: 1) включение новой фамилии студента с указанием группы, 2) добавление новой группы, 3) по заданной фамилии студента вывод на экран номера группы, в которой учится студент, если заданная фамилия соответствует нескольким номерам групп, то они все выводятся на экран.

ВАРИАНТ № 20

Реализовать информационную базу, состоящую из трёх файлов: FIO.DAT - содержит в виде символьных строк, оканчивающихся <\n>, список фамилий студентов; GROUP.DAT - содержит в виде символьных строк номера студенческих групп, которые на повторяются; LINK.IDX - содержит записи вида struct {unsigned long name_idx; //индекс записи в файле FIO.DAT; unsigned long group_idx; //индекс записи в файле GROUP.DAT}, связывающие между собой фамилию и группу. В качестве индексов используются значения позиций, с которых начинаются записи в соответствующих файлах. Записи в FIO.DAT и в LINK.IDX взаимно-однозначно соответствуют друг другу. Программа должна обеспечивать выполнение следующих функций: 1) включение новой фамилии студента с указанием группы, 2) добавление новой группы, 3) по заданному номеру группы вывод на экран всех фамилий студентов, принадлежащих группе.

ВАРИАНТ № 21

Написать программу, которая поддерживает прямой доступ к записям файла, содержащего записи переменной длины вида

<*предложение_переменной_длины* – (`char[??]`)><*длина_предложения* – (`int`)>. Логически текст состоит из набора произвольных слов, разделяемых пробелами. За последним символом предложения располагается его длина в двоичном формате типа `int`. Программа должна обеспечивать выполнение следующих действий:

1) Создание и формирование исходного файла записей, и параллельное создание индексного файла для прямого доступа к записям исходного файла по ключевым словам. Записи в индексном файле должны иметь вид: <*ключевое_слово*(`char[20]`)><*индекс*(`long`)>.

2) Построение отсортированного индексного файла в порядке, лексикографического возрастания соответствующих ключевых слов (в качестве ключевого слова использовать *первое слово* предложения). Для сортировки записей индексного файла его содержимое предварительно переписать в память и использовать функцию в стиле языка С - `qsort()`);

3) Вывод на экран предложения исходного файла по заданному с клавиатуры ключевому слову.

ВАРИАНТ № 22

Написать программу, которая реализует прямой доступ к записям файла, переменной длины вида:

<*блок_символов* (`char[]`)><*длина_блока*(`int`)>.

Программа должна обеспечивать выполнение следующих действий:

1) Формирование исходного файла и параллельное создание индексного файла (для прямого доступа к записям исходного файла) с записями вида:

<*ключ_доступа* (`int]`)><*индекс*(`long`)>,

в качестве ключа использовать длину блока.

2) Упорядочить записи в индексном файле по возрастанию ключа доступа. Для этого предварительно перенести содержимое в память и для сортировки использовать функцию в стиле языка С - `qsort()`.

3) Вывод на экран всех блоков символов исходного файла заданной с клавиатуры длины.

ВАРИАНТ № 23

Написать программу, которая поддерживает прямой доступ к файлу, содержащему записи переменной длины вида <*предложение_переменной_длины* – (`char[??]`)><*длина_предложения* – (`int`)>. Логически текст состоит из набора произвольных слов, разделяемых пробелами. За последним символом предложения располагается его длина в двоичном формате типа `int`. Программа должна обеспечивать выполнение следующих действий:

1) Формирование исходного файла и параллельное создание индексного файла с записями вида:

<*ключ_доступа* (`char[20]`)><*индекс*(`int`)>,

В качестве ключа доступа использовать *первое слово* предложения;

2) Упорядочить записи в индексном файле по убыванию ключа доступа (лексикографически). Для этого предварительно перенести содержимое индексного файла в память и для сортировки использовать функцию в стиле языка С - `qsort()`.

3) Формирование нового файла, в который необходимо записать предложения исходного файла, добавив к ним управляющий символ '\n', в порядке следования записей в индексном файле.

ВАРИАНТ № 24

Написать программу, которая поддерживает прямой доступ к файлу, содержащему записи структуры вида: `struct {char name[21];` //фамилия студента; `unsigned` mark; //средний балл;}. Программа должна обеспечивать выполнение следующих функций:

1) Формирование файла записей и создание двух индексных файлов, в первом индексы располагаются в порядке лексикографического возрастания значений поля `name` соответствующих записей, а во втором – в порядке возрастания значений поля `mark` соответствующих записей. Для этого предварительно записи для индексных файлов сформировать в памяти, отсортировать, а затем записать в индексный файл. Для сортировки использовать функцию в стиле языка С - `qsort()`.

2) Вывод на экран в лексикографическом порядке фамилий всех студентов.

3) Вывод на экран фамилий всех студентов по убыванию среднего балла.

ВАРИАНТ № 25

Написать программу, которая поддерживает прямой доступ к файлу, содержащему записи структуры вида: `struct {char name[21];` //фамилия студента; `unsigned` mark; //средний балл;}. Программа должна обеспечивать выполнение следующих функций:

1) Формирование файла записей и создание двух индексных файлов, в первом индексы располагаются в порядке лексикографического возрастания значений поля `name` соответствующих записей, а во втором – в порядке возрастания значений поля `mark` соответствующих записей. Для этого предварительно записи для индексных файлов сформировать в памяти, отсортировать, а затем записать в индексный файл. Для сортировки использовать функцию в стиле языка С - `qsort()`.

2) На основе исходного и индексных файлов сформировать два новых файла с заданными с клавиатуры именами. В первый должны войти все записи исходного файла, упорядоченные по фамилиям студентов, а во второй – по среднему баллу.

Приложение 1

ЗАДАНИЕ

Ввести с клавиатуры произвольное абстрактное предложение произвольной длины, состоящее из абстрактных слов. Слова в предложении отделяются друг от друга как минимум одним символом «пробел». Используя введённое предложение как исходное, построить новое предложение из слов исходного предложения не больших заданной длины.

Информационная модель предметной области

1. **Информационный объект СЛОВО (**тип Сущность**)**
   - *Свойства:*
     - Строка
     - Длина_Строки
   - *Процедуры*:
     - Создание_Слова
     - Получить_Слово_с_перевёрнутой_Строкой
     - Аннулирование_Слова
2. **Информационный объект СПИСОК_СЛОВ (**тип Связь**)**
   - *Свойства:*
     - Список_Слов
     - Длина_Списка_Слов
   - *Процедуры*:
     - Построение_пустого_Списка_Слов
     - Добавление_Слова_в_Список_Слов
     - Получение_из_Списка_Слова_по_порядковому_номеру
     - Построение_Предложения_по_Списку_Слов
     - Аннулирование_Списка_Слов
3. **Информационный объект ПРЕДЛОЖЕНИЕ (**тип Сущность**)**
   - *Свойства:*
     - Символьная_строка
   - *Процедуры:*
     - Ввод_Символьной_строки_и_создание_Предложения
     - Создание_Предложения_по_заданой_Символьной_строке
     - Построение_Списка_Слов_Предложения
     - Построение_Символьной_строки_по_заданному_Списку_Слов
     - Вывод_Строки_Предложения
     - Аннулирование_Предложения

Структура алгоритма программы

1. Создание Исходного_Предложения
2. Построение Списка_Слов Исходного_Предложения

3. Создание пустого Результатного_Предложения
4. Цикл построения по заданному Предложению Списка_Слов заданной длины
5. Построение Символьной_строки по заданному Списку_Слов
6. Построение результатного Предложения по заданной Символьной_строке
7. Вывод результатного Предложения.
8. Завершить работу

Приложение 2

ЗАДАНИЕ

Написать программу, которая создаёт текстовый файл и заполняет его словами, вводимыми с клавиатуры, завершающимися символом '\n'. После ввода последнего слова файл закрывается. Создать новый файл с именем, вводимым с клавиатуры, в котором слова исходного файла располагаются в лексикографическом порядке. Для сортировки слов содержимое исходного файла полностью переписывается в оперативную память. При написании программы должна быть использована стандартная функция в стиле языка С - qsort().

Объектная модель предметной области

1. **Информационный объект** – "**Слово**" (тип Сущность)

Свойства:

- Символьная_строка\n
- Размер буфера

Процедуры:

- Создать
- Аннулировать

2. **Информационный объект** – "**БуферСлов**" (тип Сущность)

Свойства:

- Буфер памяти
- Размер буфера

Процедуры:

- Создать БуферСлов заданного размера
- Отсортировать слова в БуферСлов
- Аннулировать БуферСлов

3. **Информационный объект** – "**ФайлСлов**" (тип Сущность)

Свойства:

- Имя файла
- Расширение имени
- Расположение файла в файловой системе (Путь)
- Буфер ввода/вывода слов

Процедуры:

- Создать ФайлСлов
- Открыть для записи
- Открыть для чтения
- Записать Слово
- Прочитать Слово
- Скопировать содержимое в БуферСлов
- Скопировать содержимое БуфераСлов в созданный файл

- Закрыть ФайлСлов
- Аннулировать ФайлСлов

Структура алгоритма программы

1. Создать Исходный_ФайлСлов
2. Скопировать содержимое в БуферСлов
3. Отсортировать слова в БуферСлов
4. Создать новый ФайлСлов
5. Скопировать содержимое БуферСлов в новый ФайлСлов
6. Завершить работу

Приложение 3

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА (САМАРСКИЙ УНИВЕРСИТЕТ)»

ИНСТИТУТ ИНФОРМАТИКИ И КИБЕРНЕТИКИ
Кафедра программных систем

Дисциплина
Объектно-ориентированное программирование

ОТЧЕТ

по лабораторной работе №____

<Название лабораторной работы>

Вариант №___

Выполнил: Фамилия И.О., группа № 6XXX-XXXXXXX

Проверил: Фамилия И.О.

Самара 2026